Diffusion Scores for Markov Data

A technical compendium

Exact inference on the noised chain, estimation of the transition kernel, and comparison with neural denoisers

Giovanni Mantovani
Department of Computing Sciences, Bocconi University
giovanni.mantovani@studbocconi.it

August 6, 2026

Status. Work in progress, circulated internally. This document is the long-form companion to the internal note *Diffusion scores for Markov data*. Where the note states a result, this document derives it. Chapters covering experiments still running are marked [PENDING] and will be completed as those results land; nothing marked pending is relied on elsewhere.

Who this is for. A reader who knows undergraduate probability and linear algebra and nothing about diffusion models, graphical models, or expectation maximisation. Every concept used anywhere in the project is built here from its definition. Nothing is cited in place of an explanation. Where a step is standard but non-obvious, it is done in full rather than asserted, and where an assumption is doing real work, there is a box saying what breaks without it.

Contents

Chapter 1

Orientation

1.1 What problem is being solved

A generative model is a machine that, having seen a finite collection of examples, produces new ones that look like they came from the same source. Diffusion models do this by a two-stage trick: first destroy the data by adding noise gradually until nothing is left but pure noise, then learn to run that destruction backwards.

Running it backwards requires knowing, at every intermediate noise level, which direction points back towards plausible data. That direction is called the *score*, and it turns out to be equivalent to answering a question that sounds much more ordinary:

> *Given a corrupted example, what was the clean one, on average?*

That average — the posterior mean — is the object everything in this project is about. A trained diffusion model is, in the end, a machine that estimates it.

1.2 The difficulty this project addresses

For almost every interesting distribution of data, nobody can compute that posterior mean. There is no formula. So when a neural network is trained to estimate it and makes errors, there is no way to tell whether the network was bad or the target was hard. The two are confounded, and that confound blocks a lot of questions one would like to ask about why diffusion models work.

The few distributions where the posterior mean *is* computable are mostly Gaussians, and Gaussians are unhelpful here: for them the posterior mean is a linear function of the observation, so every question about how a model exploits structure has a trivial answer.

This project takes a distribution that is

- structured enough to be interesting — the coordinates are correlated;
- non-Gaussian, so the posterior mean is genuinely nonlinear;
- and yet exactly computable.

The distribution is a **Markov chain**: a sequence in which each entry depends on the previous one and, given that, on nothing earlier. Chapter 5 defines this properly. The reason the posterior mean is computable is a structural accident that Chapter 7 explains in full, and which can be previewed in one sentence: noising each coordinate independently does not create any new dependencies between the clean coordinates, so the machinery that makes chains tractable survives the noising intact.

1.3 The three questions

1. **What does exact inference buy?** If one has the true Markov chain, one can compute the posterior mean exactly. How much better is that than the standard Gaussian approximation, and at which noise levels does the difference live? (Chapters 7, 8.)

2. **Can the chain be learned?** In reality the chain is unknown. Can its transition rule be estimated from noisy observations alone — never seeing a clean example — and how expensive is that? (Later chapters, in progress.)

3. **How does this compare with a neural network?** Given the same data, does a structured estimator beat a trained denoiser, and does any advantage in estimating the posterior mean translate into better generated samples? (Later chapters, in progress.)

The third question has an answer that surprised us, and it is the reason the accompanying note is framed the way it is. It is stated in the note and derived here once the supporting machinery is in place.

1.4 How to read this document

Chapters 2–4 build the diffusion setting from probability first principles. Chapters 5–7 build the graphical-model machinery and prove the exactness result. Chapter 8 treats the Gaussian case, which is both the sanity check and the baseline. Chapter 9 explains what an actual implementation has to do and where its errors come from.

Three kinds of box appear throughout:

Intuition

Explains why a result is what it is, in words, before or after the algebra. Skippable if the algebra was enough.

What goes wrong without this

Names the concrete failure that occurs if an assumption is dropped. These are not hypothetical — most of them describe mistakes actually made during this project and caught later.

In the code

Points at the exact file and function implementing the object just defined. Paths are relative to `research/nongaussian-bp/` in the repository.

1.5 Map of the code

Everything referenced in this document lives under `research/nongaussian-bp/`.

Path	Contents
`src/priors.py`	The data-generating models: `GaussianAR1`, `LaplaceAR1`, `StudentTAR1`, `UniformAR1`, `GaussianMixtureAR1`. Each provides `sample` and `log_transition_matrix`.
`src/bp_grid.py`	The inference recursion. `make_grid`, `grid_bp` (one sequence, with evidence), `grid_bp_batch` (many sequences, device-parameterised).
`src/bp_gaussian.py`	The closed-form Gaussian recursion in information form.
`src/exact_scores.py`	Closed-form Gaussian posterior mean and score, used as ground truth.
`src/em.py`	The expectation step, the sufficient statistic, and the estimation driver.
`src/kernels.py`	Parametrised transition families and their maximisation steps.
`src/denoiser.py`	A common interface over inference-based and network-based denoisers, plus the network training loop.
`src/reverse.py`	Reverse-time integration: SDE and probability-flow ODE.
`src/sample_metrics.py`	Statistics for comparing generated data against the truth.
`src/backend.py`	CPU/GPU array-module dispatch for the recursion.
`experiments/`	`exp_01`–`exp_17`, one file per experiment, each self-describing.
`tests/`	Validation. Several tests here are the evidence for claims made in this document and are cited as such.

In the code

Nothing in this document is stated without a pointer to where it is realised. If a derivation here has no corresponding **In the code** box, that is deliberate and means the result is used only on paper.

Chapter 2

Probability, conditioning, and estimation

This chapter fixes the small amount of probability the rest of the document uses. A reader comfortable with conditional expectation and the orthogonality principle can skip to Chapter 3, pausing only at §2.3, whose result is used constantly.

2.1 Densities, joints, conditionals

We work with random vectors on $\mathbb{R}^n$ with densities. Write $P(a)$ for the density of a, $P(a, x)$ for a joint density, and

$$P(a \mid x) \;=\; \frac{P(a,x)}{P(x)}, \qquad P(x) \;=\; \int \mathrm{d}a\; P(a,x), \tag{2.1}$$

for the conditional. The second equation — integrating out what one does not care about — is called *marginalisation*, and it is the operation that will turn out to be expensive and that all the machinery in Chapter 7 exists to make cheap.

Bayes' rule is the same statement rearranged:

$$P(a \mid x) \;=\; \frac{P(x \mid a)\,P(a)}{P(x)} \;\propto\; \underbrace{P(x \mid a)}_{\text{likelihood}}\;\underbrace{P(a)}_{\text{prior}}. \tag{2.2}$$

The proportionality is in a: the denominator $P(x)$ does not depend on a and serves only to normalise. Almost every calculation in this document works with the unnormalised product and restores the constant at the end, because the constant is usually the expensive part.

Intuition

The prior says what clean data look like before seeing anything. The likelihood says how plausible the observation is for each candidate clean value. Their product, renormalised, is what one should believe after seeing the observation. In our setting the prior is the Markov chain and the likelihood is the noise model.

2.2 Expectation and conditional expectation

The expectation of $g(a)$ is $\mathbb{E}[g(a)] = \int \mathrm{d}a\, P(a)g(a)$. The *conditional* expectation given x is the same integral against the conditional density,

$$\mathbb{E}[g(a) \mid x] \;=\; \int \mathrm{d}a\; P(a \mid x)\, g(a). \tag{2.3}$$

Following the notation of the diffusion literature we are matching, we write the conditional mean of the clean signal with angle brackets,

$$\langle a \rangle_{x,t} \;:=\; \mathbb{E}[a \mid x, t] \;=\; \int \mathrm{d}a \; P(a \mid x, t)\, a, \tag{2.4}$$

where the subscript records that the conditioning is on the observation x at noise level t. Componentwise, $\langle a_i \rangle_{x,t}$ is the posterior mean at site i.

A fact used repeatedly: conditional expectation is a *function of* x, not a number. It is the function that will be estimated, whether by exact computation or by a neural network.

2.3 The Bayes-optimal estimator under squared loss

This is the single most-used result in the document, so it is proved rather than quoted.

Proposition 2.1 (Orthogonality)**.** *Let a have finite second moment. For any u that is a function of x alone,*

$$\mathbb{E}\big[\|a - u\|^2 \,\big|\, x\big] \;=\; \mathbb{E}\big[\|a - \langle a \rangle_{x,t}\|^2 \,\big|\, x\big] \;+\; \|u - \langle a \rangle_{x,t}\|^2. \tag{2.5}$$

Consequently $\langle a \rangle_{x,t}$ minimises the conditional mean squared error, and hence also the unconditional one.

Proof. Insert and subtract $\langle a \rangle_{x,t}$:

$$\|a-u\|^2 = \|(a-\langle a\rangle_{x,t}) + (\langle a\rangle_{x,t}-u)\|^2 = \|a-\langle a\rangle_{x,t}\|^2 + 2\langle a-\langle a\rangle_{x,t},\, \langle a\rangle_{x,t}-u\rangle + \|\langle a\rangle_{x,t}-u\|^2.$$

Take $\mathbb{E}[\cdot \mid x]$. In the cross term, $\langle a \rangle_{x,t} - u$ is a function of x only, so it is constant under the conditional expectation and can be pulled out:

$$\mathbb{E}[\langle a - \langle a\rangle_{x,t},\, \langle a\rangle_{x,t} - u\rangle \,|\, x] = \big\langle \mathbb{E}[a \mid x] - \langle a\rangle_{x,t},\; \langle a\rangle_{x,t} - u \big\rangle = 0,$$

since $\mathbb{E}[a \mid x] = \langle a \rangle_{x,t}$ by definition. The remaining two terms give (2.5). The first is independent of u and the second is non-negative and vanishes only at $u = \langle a \rangle_{x,t}$. Taking a further expectation over x preserves the minimiser. □

Two consequences deserve emphasis.

There is an irreducible floor. The first term of (2.5) does not depend on the estimator. It is the posterior variance, and no denoiser — exact, learned, or otherwise — can go below it. A reported denoising loss is therefore uninterpretable without knowing that floor.

What goes wrong without this

Early in this project a test asserted that a network should achieve a 40% reduction in denoising loss. That target was arithmetically impossible: with the floor at 7.4 and the loss of the trivial predictor $f \equiv 0$ at 12.06, the largest achievable reduction was 39%. The test could never pass however well the network trained. The fix was to report the *fraction of the achievable reduction* rather than the raw one.

In the code

`src/sample_metrics.py` → `bayes_risk` computes the floor; `src/sample_metrics.py` → `pointwise_ladder` returns `fraction_achievable_captured` alongside the raw loss for exactly this reason.

Optimality is loss-specific. Proposition 2.1 is about squared error. Under absolute error the optimal estimator is the posterior median; under 0–1 loss it is the mode. Diffusion training uses squared error, which is why the posterior *mean* is the relevant functional — not because it is intrinsically the right summary of a posterior.

2.4 Linear estimators and LMMSE

Sometimes one restricts to estimators of the form $\widehat{a} = Bx$ for a matrix B. The best such estimator under squared loss is the *linear minimum mean squared error* estimator, and it depends on the joint distribution only through second moments.

Proposition 2.2. *Let a, x have zero mean and finite second moments, with $\mathrm{Cov}(x)$ invertible. Then*

$$\widehat{a}_{\mathrm{LMMSE}} \;=\; \mathrm{Cov}(a,x)\;\mathrm{Cov}(x)^{-1}x \tag{2.6}$$

minimises $\mathbb{E}\|a - Bx\|^2$ over matrices B.

Proof. Expand $\mathbb{E}\|a - Bx\|^2 = \mathrm{tr}\big(\mathrm{Cov}(a) - 2B\,\mathrm{Cov}(x,a) + B\,\mathrm{Cov}(x)B^{\top}\big)$ and differentiate with respect to B: $-2\,\mathrm{Cov}(a,x) + 2B\,\mathrm{Cov}(x) = 0$, so $B = \mathrm{Cov}(a,x)\,\mathrm{Cov}(x)^{-1}$. The objective is convex in B because $\mathrm{Cov}(x) \succ 0$, so the stationary point is the minimum. □

Intuition

LMMSE is what one gets from knowing only the covariance structure. If the joint distribution happens to be Gaussian, the conditional mean is already linear and LMMSE coincides with the Bayes-optimal estimator. Otherwise LMMSE is strictly worse, and the gap is exactly the value of the higher-order structure. Chapter 8 shows that a natural and widely used approximation to exact inference silently computes LMMSE, which is why the distinction matters so much here.

2.5 Two ways a distribution can be "the data"

This distinction causes a common error, so it is fixed now.

The *population* distribution P_0 is the true source of the data. One never has it — one has M samples $a^1, \dots, a^M$ drawn from it. Those samples define the *empirical* distribution

$$\widehat{P}_0 \;=\; \frac{1}{M}\sum_{\mu=1}^{M} \delta_{a^\mu}, \tag{2.7}$$

a sum of point masses. It is a perfectly good probability distribution, and it is not the population one.

Correspondingly there are two versions of any quantity defined from a distribution: the population posterior mean, computed under P_0, and the empirical posterior mean, computed under $\widehat{P}_0$. A training procedure that minimises an average over the observed samples is minimising the *empirical* objective, and its unrestricted optimum is the *empirical* posterior mean.

What goes wrong without this

It is tempting — and wrong — to say that the population objective is minimised by the empirical score. The population objective is minimised by the population posterior mean, by Proposition 2.1; it is the *empirical* objective whose optimum is the empirical posterior mean. The distinction is the whole content of the memorisation discussion in §4.3, and conflating the two makes that discussion incoherent. An earlier draft of the accompanying

note contained exactly this error.

Chapter 3

The forward diffusion

3.1 Destroying data on purpose

A diffusion model is built around a process that takes a data point and gradually turns it into noise. The requirements are modest: the process should be easy to simulate, its end-state should be easy to sample from, and — crucially — the distribution at every intermediate time should be something we can reason about.

The Ornstein–Uhlenbeck (OU) process satisfies all three. Following the convention of the notes we are matching (Mézard, 2025), it is the stochastic differential equation

$$\frac{\mathrm{d}x}{\mathrm{d}t} \;=\; -x + \sqrt{2}\,\eta(t), \qquad x(0) = a, \tag{3.1}$$

where η is Gaussian white noise with $\langle \eta(t) \rangle = 0$ and $\langle \eta_i(t)\eta_j(t') \rangle = \delta_{ij}\delta(t-t')$.

Two forces act. The term $-x$ pulls towards the origin; the noise pushes outwards. At long times they balance and the process forgets where it started.

3.2 Solving it

Equation (3.1) is linear, so it can be integrated explicitly. Multiply by the integrating factor e^t:

$$\frac{\mathrm{d}}{\mathrm{d}t}\big(e^t x\big) = e^t\left(\frac{\mathrm{d}x}{\mathrm{d}t} + x\right) = \sqrt{2}\,e^t \eta(t).$$

Integrating from 0 to t and using $x(0) = a$,

$$x(t) \;=\; e^{-t}a + \sqrt{2}\int_0^t \mathrm{d}\tau\; e^{-(t-\tau)}\eta(\tau). \tag{3.2}$$

The remaining integral is a sum of independent Gaussian contributions, hence Gaussian, with mean zero and variance

$$2\int_0^t \mathrm{d}\tau\; e^{-2(t-\tau)} = 2\,e^{-2t}\int_0^t \mathrm{d}\tau\, e^{2\tau} = 1 - e^{-2t}.$$

So the whole solution can be written with a single standard Gaussian z:

$$\boxed{x(t) \;=\; e^{-t}a + \sqrt{\Delta_t}\,z, \qquad \Delta_t \;=\; 1 - e^{-2t}, \qquad z \sim \mathcal{N}(0, I)\,.} \tag{3.3}$$

This one equation is used everywhere below. Read it as: *the signal is scaled down by e^{-t} and independent Gaussian noise of variance Δ_t is added.*

Intuition

The two coefficients move in opposite directions and are tied together. At $t = 0$ we have $e^{-t} = 1$, $\Delta_t = 0$: the data, untouched. As $t \to \infty$, $e^{-t} \to 0$ and $\Delta_t \to 1$: pure standard Gaussian noise, with the data completely gone. The tie $e^{-2t} + \Delta_t = 1$ means the total variance is preserved when the data has unit variance, so nothing blows up or collapses along the way.

In the code

Noising is applied wherever an experiment needs it, always as `alpha * a + sqrt(delta) * rng.standard_normal(...)` with `alpha = exp(-t)`, `delta = 1 - exp(-2t)`. See for instance `experiments/exp_16_sampling_validation.py` → `fit_arms`, and `src/noising.py` → `alpha_delta` for the coefficient pair.

3.3 The noised distribution P_t

Equation (3.3) describes one trajectory. What matters is the distribution of $x(t)$ when a is drawn from P_0. Since x given a is Gaussian with mean $e^{-t}a$ and covariance $\Delta_t I$, marginalising over a gives

$$\boxed{P_t(x) \;=\; \int \mathrm{d}a\; P_0(a)\, \frac{1}{(2\pi\Delta_t)^{n/2}} \exp\left[-\frac{\|x - e^{-t}a\|^2}{2\Delta_t}\right].} \tag{3.4}$$

That is: P_t is P_0 convolved with a Gaussian, after a rescaling. Everything about the forward process is contained in this formula.

Intuition

Convolution with a Gaussian is smoothing. Whatever sharp features P_0 has — spikes, edges, corners — get blurred, and blurred more as t grows. This is why the problem gets easier at large t (everything looks Gaussian) and why the interesting behaviour is at small and intermediate t, where the original structure is still partly visible.

3.4 Why the coordinatewise structure of the noise matters

One feature of (3.3) is so important to this project that it deserves isolating now, even though its consequence only becomes visible in Chapter 7.

The noise added to coordinate i is independent of the noise added to coordinate j. So the conditional density of the whole observation factorises over sites:

$$P(x \mid a, t) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\Delta_t}} \exp\left[-\frac{(x_i - e^{-t}a_i)^2}{2\Delta_t}\right] \;=\; \prod_{i=1}^{n} \ell_t(x_i \mid a_i). \tag{3.5}$$

Each factor $\ell_t(x_i \mid a_i)$ involves *one* clean coordinate and *one* observation.

What goes wrong without this

If the forward noise were correlated across coordinates — say with covariance Σ_z instead of I — then (3.5) would not factorise, the likelihood would couple different a_i directly, and the graphical structure exploited in Chapter 7 would be destroyed. Everything in this project depends on the noise being coordinatewise. It is not a cosmetic modelling choice.

3.5 Reversing the process

Generation runs the process backwards: start from noise at a large time and integrate towards $t = 0$. The reverse-time dynamics of (3.1) are

$$\frac{\mathrm{d}x}{\mathrm{d}\tau} \;=\; x + 2\,S(x,\tau) + \eta(\tau), \qquad S(x,t) := \nabla_x \log P_t(x), \tag{3.6}$$

where τ runs backwards in t. The field S is the *score*, and it is the only part of (3.6) that depends on the data distribution. Everything else is known.

So the entire modelling problem is: *estimate the score*. Chapter 4 shows that this is the same as estimating the posterior mean.

In the code

`src/reverse.py` → `reverse_sde` integrates (3.6) by Euler–Maruyama; `src/reverse.py` → `probability_flow_ode` integrates the deterministic counterpart with Heun's method. Both accept any score function with signature `(X, t) -> S`, which is how the different estimators are compared on equal footing. `src/reverse.py` → `time_grid` builds the geometric time discretisation.

What goes wrong without this

Integration cannot be carried to $t = 0$. As $\Delta_t \to 0$ the score (4.2) carries a factor $1/\Delta_t$ and diverges. Every implementation stops at some $t_{\min} > 0$, and what it produces is therefore a sample of $P_{t_{\min}}$, *not* of P_0. Chapter 9 returns to this; it caused two successive errors in this project's own measurements before it was handled correctly.

Chapter 4

The score is the posterior mean

This chapter proves the identity that makes the whole project possible: the score, which is what the reverse process needs, is a rescaling of the posterior mean, which is what inference computes. They are the same problem.

4.1 The derivation

Start from (3.4) and differentiate with respect to x. The only x-dependence is in the Gaussian factor, and

$$\nabla_x \exp\left[-\frac{\|x-e^{-t}a\|^2}{2\Delta_t}\right] = -\frac{x-e^{-t}a}{\Delta_t}\,\exp\left[-\frac{\|x-e^{-t}a\|^2}{2\Delta_t}\right],$$

so

$$\nabla_x P_t(x) = \int \mathrm{d}a\; P_0(a)\;\left(-\frac{x-e^{-t}a}{\Delta_t}\right)\frac{1}{(2\pi\Delta_t)^{n/2}}\exp\left[-\frac{\|x-e^{-t}a\|^2}{2\Delta_t}\right]. \tag{4.1}$$

Now divide by $P_t(x)$. By Bayes' rule (2.2), the quantity

$$\frac{P_0(a)\,(2\pi\Delta_t)^{-n/2}\exp\!\big[-\|x-e^{-t}a\|^2/2\Delta_t\big]}{P_t(x)}$$

is exactly the posterior density $P(a \mid x,t)$: numerator is prior times likelihood, denominator is the evidence. So (4.1) becomes

$$S(x,t) = \frac{\nabla_x P_t(x)}{P_t(x)} = \int \mathrm{d}a\; P(a\mid x,t)\left(-\frac{x-e^{-t}a}{\Delta_t}\right) = -\frac{x}{\Delta_t} + \frac{e^{-t}}{\Delta_t}\int \mathrm{d}a\; P(a\mid x,t)\,a,$$

and the remaining integral is the posterior mean. Hence

$$\boxed{S(x,t) \;=\; \nabla_x \log P_t(x) \;=\; -\frac{x-e^{-t}\,\langle a\rangle_{x,t}}{\Delta_t}.} \tag{4.2}$$

When is the differentiation legitimate? Interchanging ∇_x and $\int \mathrm{d}a$ requires a dominating function. The integrand's gradient is bounded in absolute value by

$$P_0(a)\,\frac{|x-e^{-t}a|}{\Delta_t}\,\frac{e^{-\|x-e^{-t}a\|^2/2\Delta_t}}{(2\pi\Delta_t)^{n/2}},$$

and for x in a neighbourhood of any fixed point this is dominated by $C(1+\|a\|)P_0(a)$ for a constant C depending on Δ_t and the neighbourhood, since the Gaussian factor is bounded and decays. That dominating function is integrable exactly when P_0 has a finite first moment. Dominated convergence then applies. All the priors used here have moments of every order, so the condition is not restrictive.

Intuition

The formula says something concrete. Rearranged, $\langle a \rangle_{x,t} = e^t(x + \Delta_t\, S(x,t))$: to guess the clean signal, take the observation, step along the score by an amount proportional to how noisy things are, and undo the shrinkage. At small t, Δ_t is small and the correction is tiny — the observation is already almost the answer. At large t, $\Delta_t \to 1$ and the correction does all the work.

In the code

`src/bp_grid.py` → `score_from_posterior_mean` and `src/denoiser.py` → `score_from_mean` both implement (4.2). Every arm in every comparison converts a posterior mean to a score through one of these, so no arm can gain an advantage from a different convention.

4.2 Consequences worth stating

4.2.1 Estimating the score and denoising are one problem

There is no separate "score model". A denoiser and a score model are the same object in two coordinate systems. This is why diffusion models can be trained by a regression loss on clean signals and then used to drive a differential equation — the training target and the sampling requirement are related by (4.2).

4.2.2 The two error measures differ by a known factor

Applying (4.2) to two different denoisers $\widehat{m}$ and $m^\star$ and subtracting, the x terms cancel:

$$\widehat{S} - S^\star = \frac{e^{-t}}{\Delta_t}\,(\widehat{m} - m^\star)\,, \qquad \text{so} \qquad \|\widehat{S} - S^\star\| = \frac{e^{-t}}{\Delta_t}\,\|\widehat{m} - m^\star\|. \tag{4.3}$$

Score error and posterior-mean error are therefore *the same measurement* up to the factor e^{-t}/Δ_t. That factor is not mild: it runs from about 6.0 at $t = 0.08$ to about 0.09 at $t = 2.4$, a spread of roughly 65.

What goes wrong without this

Reporting only a score error silently weights the low-noise end of the schedule about sixty-five times more heavily than the high-noise end, relative to reporting a posterior-mean error. Two methods can therefore be ranked differently by the two metrics without either metric being wrong. This project reports both, and states the factor, so that a reader can convert. Earlier work here reported score error alone.

In the code

`src/metrics.py` → `score_mean_identity_residual` checks (4.3) holds to machine precision for any pair of denoisers — a large residual means an implementation bug, not a modelling difference. `src/sample_metrics.py` → `pointwise_ladder` emits `score_reweighting` next to every error it reports. Verified in `tests/test_sample_metrics.py` → `test_score_reweighting_matches_the_tweedie_factor` and `test_reweighting_spans_the_claimed_range_across_the_schedule`.

4.2.3 The Gaussian case, as a check

If $P_0 = \mathcal{N}(0, \Sigma_0)$ then x is Gaussian with covariance $\Sigma_t = e^{-2t}\Sigma_0 + \Delta_t I$, and the posterior mean is linear:

$$\langle a \rangle_{x,t} = e^{-t}\Sigma_0 \Sigma_t^{-1} x, \qquad S(x,t) = -\Sigma_t^{-1} x. \tag{4.4}$$

The second follows by differentiating $\log P_t(x) = -\frac{1}{2} x^\top \Sigma_t^{-1} x + \text{const}$ directly, and consistency with (4.2) is a one-line check that we use as a test of the whole pipeline.

In the code

`src/exact_scores.py` → `exact_gaussian_posterior_mean` and `src/exact_scores.py` → `precision_matrix_score` implement (4.4). These are the ground truth against which the general machinery of Chapter 7 is validated; see `tests/test_score_mean_error_identity.py`.

4.3 Three scores, not one

Equation (4.2) defines the score of a distribution. Which distribution one has in mind changes what the object is, and three different choices appear in the literature under the same word.

The population score. Take P_0 to be the true data distribution. This is the score that the reverse process would need in order to generate genuinely new data distributed as P_0. Nobody has it, because nobody has P_0.

The empirical score. Take $\widehat{P}_0$ from (2.7), the sum of point masses at the training examples. Its noised version is a Gaussian mixture centred on those examples, and its posterior mean is a softmax-weighted average of them:

$$\widehat{m}(x,t) = \frac{\sum_\mu a^\mu \exp\!\left[-\|x - e^{-t}a^\mu\|^2 / 2\Delta_t\right]}{\sum_\nu \exp[-\|x - e^{-t}a^\nu\|^2 / 2\Delta_t]}. \tag{4.5}$$

Note what this returns as $t \to 0$: the nearest training example. Exact reversal under this score reproduces the training set and nothing else.

The learned score. What a network with finite capacity, finite data and a particular optimiser actually produces.

What goes wrong without this

The chain of reasoning that matters, stated carefully:

1. Training minimises an average over the observed samples — the *empirical* objective.
2. Its unrestricted minimiser over all measurable functions is the *empirical* posterior mean (4.5), by Proposition 2.1 applied under $\widehat{P}_0$.
3. Exact reversal under that score returns $\widehat{P}_0$ — the training set.
4. Therefore a model that perfectly attained the optimum of its own training objective would be useless as a generative model.

That trained models nonetheless generalise is a statement about what stops them from attaining that optimum — capacity, architecture, optimisation, regularisation — not about the objective. It is *not* correct to say the population objective is minimised by the empirical score; the population objective is minimised by the population posterior mean.

Intuition

This is the tension that motivates wanting a setting where the *population* score is computable. If one can compute it, one can ask how far a trained network is from it, and separate "the network is bad" from "this target is hard". That is what the Markov chain of Chapter 5 provides.

Chapter 5

The data model: a Markov chain

5.1 Definition

A sequence $a = (a_1, \dots, a_n)$ is a *first-order Markov chain* if, given the present, the future is independent of the past:

$$P(a_i \mid a_{i-1}, a_{i-2}, \dots, a_1) \;=\; P(a_i \mid a_{i-1}) \qquad \text{for all } i. \tag{5.1}$$

Applying the chain rule of probability and then (5.1) repeatedly,

$$\boxed{P_0(a) \;=\; \mu(a_1) \prod_{i=2}^{n} W(a_i \mid a_{i-1}),} \tag{5.2}$$

where μ is the law of the first site and W is the *transition kernel*. We take W homogeneous — the same rule at every step — which will matter in Chapter 7 and again when we estimate it.

Intuition

The factorisation is the whole point. A general density on $\mathbb{R}^n$ is an unmanageable object. Equation (5.2) says this one is built from $n-1$ copies of a single two-variable function. Everything tractable about the model descends from that.

5.2 The specific family used here

We use additive transitions with a linear autoregression,

$$a_i \;=\; \rho\, a_{i-1} + \varepsilon_i, \qquad \varepsilon_i \sim \varphi \text{ i.i.d.}, \qquad W(a' \mid a) = \varphi(a' - \rho a), \tag{5.3}$$

with $|\rho| < 1$, and φ an *innovation density* of mean zero and variance q.

5.2.1 Stationarity and the covariance

Suppose a_1 is drawn so that the chain is stationary, i.e. $\operatorname{Var}(a_i) = \sigma^2$ for all i. Taking variances in (5.3) and using independence of ε_i from a_{i-1},

$$\sigma^2 = \rho^2 \sigma^2 + q \qquad \Longrightarrow \qquad \sigma^2 = \frac{q}{1-\rho^2}.$$

Choosing $q = 1 - \rho^2$ therefore gives unit marginal variance. For the covariance, multiply (5.3) by a_{i-k} and take expectations; since ε_i is independent of everything earlier,

$$\operatorname{Cov}(a_i, a_{i-k}) = \rho \operatorname{Cov}(a_{i-1}, a_{i-k}) = \cdots = \rho^k \sigma^2,$$

so with the normalisation above

$$\boxed{(\Sigma_0)_{ij} \;=\; \mathrm{Cov}(a_i, a_j) \;=\; \rho^{|i-j|}.} \tag{5.4}$$

Intuition

Correlation decays geometrically with separation, with ρ setting the range. At $\rho = 0.85$, sites five apart still share $\rho^5 \approx 0.44$ of their variation; sites twenty apart share 0.04. There is a well-defined correlation length, and it will reappear in Chapter 8 as the scale controlling how much context a denoiser needs.

5.2.2 The crucial degree of freedom

Here is the feature that makes this family a good experimental probe, and it is worth dwelling on.

Equation (5.4) depends on φ *only through its variance* q. Everything else about the innovation — whether it is Gaussian, heavy-tailed, bounded, bimodal — leaves no trace in the covariance whatsoever.

So one can build a family of data distributions that are

- *identical* in every second-order statistic, and
- *different* in every higher-order one.

family	innovation	excess kurtosis	covariance
`GaussianAR1`	$\mathcal{N}(0, q)$	0	$\rho^{\lvert i-j\rvert}$
`LaplaceAR1`	Laplace$(0, b)$, $2b^2 = q$	$+3$	$\rho^{\lvert i-j\rvert}$
`StudentTAR1`	scaled t_ν	$6/(\nu-4)$	$\rho^{\lvert i-j\rvert}$
`UniformAR1`	uniform on $[-c, c]$	-1.2	$\rho^{\lvert i-j\rvert}$
`GaussianMixtureAR1`	symmetric two-component	< 0, bimodal	$\rho^{\lvert i-j\rvert}$

Intuition

This is the experimental design in one table. Any method that uses only second-order information — and Chapter 8 shows that a very standard approximation does exactly that — cannot tell these five apart, by construction. Any difference in performance across the row is therefore attributable to higher-order structure and nothing else. There is no confound to argue about.

In the code

`src/priors.py` → `GaussianAR1`, `LaplaceAR1`, `StudentTAR1`, `UniformAR1`, `GaussianMixtureAR1`. Each is a frozen dataclass holding only ρ, with `q`, `sample`, `log_transition_matrix` and `innovation_excess_kurtosis` as properties. The variance matching of §5.2.2 is built into each class rather than imposed by the caller, so the families cannot accidentally be compared at mismatched covariance.

5.3 Why this is a reasonable model of anything

Two defences, and it is worth being honest that they are defences rather than derivations.

Sequential data has temporal coherence. Diffusion models are increasingly applied to video and other sequential data, where consecutive frames are strongly dependent. A first-order Markov chain is the simplest non-trivial model of that dependence. It is not a model *of* video; it is a model of the property that makes video different from a bag of independent images.

It is a controllable probe. The stronger justification is methodological. The set of distributions for which the population score is computable is small, and every member of it is a modelling compromise. This one buys genuine nonlinearity of the score — which Gaussians do not have — at the price of a strong structural assumption. Given that the aim is to study how estimators exploit structure, having structure one can dial is the point.

What goes wrong without this

The model assumes *exact* first-order Markov structure. If the data has longer-range dependence, an estimator built on (5.2) is misspecified, and no amount of data repairs a misspecification. This is the single largest limitation of everything that follows, and it is not mitigated anywhere in this document.

5.4 Sampling from the chain

Generating data is direct: draw a_1 from μ, then iterate (5.3). There is no rejection, no burn-in, no approximation.

In the code

Each prior's `sample(rng, n)` returns *one* sequence of length `n`, built by the recursion above with a_1 drawn standard normal. Batches are formed by stacking, e.g. `experiments/exp_16_sampling_validation.py` → `sample_chains`. The single-sequence signature is a deliberate simplicity, not an oversight; batching is the caller's business.

Remark 5.1. Setting $a_1 \sim \mathcal{N}(0,1)$ makes the chain exactly stationary only for the Gaussian family, where the stationary law is $\mathcal{N}(0,1)$. For the others the first site is very slightly off the stationary distribution and the chain relaxes to it within a few sites. Since $\rho = 0.85$ gives a relaxation of ρ^k, the effect is below 10^{-3} by site ~ 40, and all reported statistics either use interior sites or average over the sequence. The alternative — sampling a_1 from the exact stationary law of each family — would require a fixed-point computation per family for no measurable gain.

Chapter 6

Graphical models and factor graphs

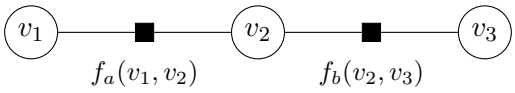
Chapter 5 gave a factorised density. This chapter turns factorisation into a picture, because the picture is what makes the key result of Chapter 7 obvious rather than surprising.

6.1 Factor graphs

A *factor graph* represents a density that is a product of local terms,

$$P(v_1, \ldots, v_N) \;\propto\; \prod_{\alpha} f_\alpha(v_{\partial\alpha})\,, \tag{6.1}$$

where each factor f_α depends on a subset $\partial\alpha$ of the variables. The graph is bipartite: a circle for each variable, a square for each factor, and an edge between them whenever the variable appears in that factor.

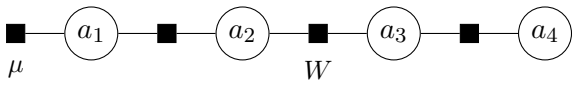


Intuition

The graph records *which variables interact directly*. Two variables not sharing a factor influence each other only through intermediaries. That is the entire content of the representation, and it is what will let us reorganise an intractable integral.

6.2 The Markov chain as a factor graph

For (5.2), the factors are $\mu(a_1)$ and one $W(a_i \mid a_{i-1})$ per consecutive pair. Each transition factor touches exactly two variables, adjacent ones, so the graph is a path:



6.3 Trees, and why they are special

Definition 6.1. A factor graph is a *tree* if it contains no cycles: between any two nodes there is exactly one path.

A path graph is a tree. This is the structural fact everything rests on.
The reason trees are special is the *separator* property.

Proposition 6.2 (Separation)**.** *Let the graph be a tree and let a_i be a variable node. Removing a_i disconnects the graph into components. Conditional on a_i, the variables in different components are independent.*

Proof. Because the graph is a tree, every factor involves variables lying in $\{a_i\}$ together with exactly one component — if a factor touched two components it would create a second path between them, hence a cycle. Group the factors by component: $P \propto \prod_c g_c\big(a_i, v^{(c)}\big)$ where $v^{(c)}$ are the variables of component c. Fixing a_i, the density in the remaining variables factorises across components, which is independence. □

Intuition

On a chain, this says: if you know a_i, then knowing anything to its left tells you nothing further about anything to its right. All the influence between the two sides is routed through a_i. That is why one can summarise everything to the left of site i in a single function of a_i — there is nothing else about the left that the right needs to know.

What goes wrong without this

The proof used acyclicity twice. On a graph with a cycle, a factor *can* join two components, the density does not factorise on conditioning, and the summary function is no longer sufficient. Belief propagation applied anyway ("loopy BP") double-counts information travelling around the cycle: a message leaving a node returns to it having been treated as independent evidence. It may still work well in practice, but it is no longer exact, and none of the guarantees in Chapter 7 survive.

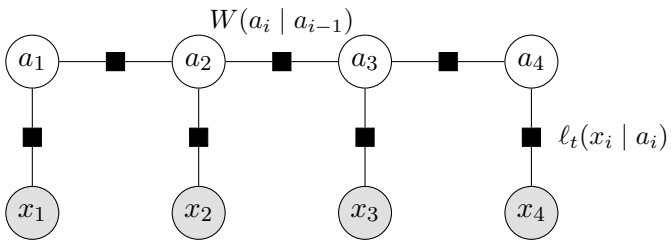
6.4 Conditioning on the observations

Now the step that makes the project work.

We observe x, related to a by (3.3). By Bayes' rule the posterior is prior times likelihood, and by (3.5) the likelihood factorises over sites:

$$P(a \mid x, t) \;\propto\; \underbrace{\mu(a_1) \prod_{i=2}^{n} W(a_i \mid a_{i-1})}_{\text{prior: chain}} \cdot \underbrace{\prod_{i=1}^{n} \ell_t(x_i \mid a_i)}_{\text{likelihood: unary}} . \tag{6.2}$$

Every likelihood factor touches *one* latent variable. In factor-graph terms it is a pendant node hanging off a_i and connected to nothing else.



Proposition 6.3. *The factor graph of the posterior (6.2) is a tree.*

Proof. The latent variables and transition factors form a path, which is acyclic. Each likelihood factor has degree one among latent variables, so attaching it adds a leaf and cannot create a cycle. The observations are fixed, not variables. □

Intuition

This is the whole trick, and it is worth saying plainly why it is not obvious. Conditioning usually *creates* dependence — explaining away is the standard example. Here it does not, because each observation is generated from a single latent variable independently of all others. So conditioning reweights the node potentials and leaves the edge set alone. The posterior is a different chain, not a non-chain.

What goes wrong without this

This fails the moment the noise is correlated across coordinates. Then $P(x \mid a)$ does not factorise, the likelihood becomes a factor touching several a_i simultaneously, and the posterior graph acquires edges — generally becoming dense and losing tractability entirely. The coordinatewise structure of the OU noise (§3.4) is doing essential work here.

6.5 What we have bought

Marginalising (6.2) naively to get $P(a_i \mid x, t)$ means integrating over $n-1$ variables — exponential in the sequence length if done by brute force on a discretisation. Proposition 6.3 says the posterior has the structure that makes this cost linear instead. Chapter 7 carries that out.

Chapter 7

Belief propagation

Chapter 6 showed the posterior is a tree. This chapter turns that into an algorithm and proves it exact.

7.1 The problem

We want $P(a_i \mid x, t)$ for each i, from

$$P(a \mid x, t) \;\propto\; \mu(a_1) \prod_{i=2}^{n} W(a_i \mid a_{i-1}) \prod_{i=1}^{n} \ell_t(x_i \mid a_i). \tag{7.1}$$

By definition this means integrating out the other $n-1$ variables. Discretising each variable to K values and summing naively costs K^{n-1} operations — at $K = 401$, $n = 32$ that is about 10^{80}, so the naive route is not merely slow but impossible.

7.2 The idea: factor out what does not depend on the summation variable

Consider a small case, $n = 3$, and write $\psi_i(a_i) := \ell_t(x_i \mid a_i)$. We want $P(a_3 \mid x)$, so we integrate over a_1 and a_2:

$$P(a_3 \mid x) \propto \psi_3(a_3) \int \mathrm{d}a_2 \int \mathrm{d}a_1 \; \mu(a_1)\psi_1(a_1)\, W(a_2 \mid a_1)\, \psi_2(a_2)\, W(a_3 \mid a_2).$$

The inner integral over a_1 involves only the first three factors — $W(a_3 \mid a_2)$ does not contain a_1, so it can be pulled out. Define

$$\overrightarrow{m}_2(a_2) := \int \mathrm{d}a_1 \; \mu(a_1)\, \psi_1(a_1)\, W(a_2 \mid a_1),$$

a function of a_2 alone. Then

$$P(a_3 \mid x) \propto \psi_3(a_3) \int \mathrm{d}a_2 \; \overrightarrow{m}_2(a_2)\, \psi_2(a_2)\, W(a_3 \mid a_2) =: \psi_3(a_3)\, \overrightarrow{m}_3(a_3).$$

Intuition

Two integrals became two *nested* integrals, each over one variable. The saving is the whole algorithm: instead of a single sum over a grid of size K^2, we do two sums of size K each, reusing the first result. That is dynamic programming, and it works because the factorisation lets the summand split.

7.3 The general recursion

Generalising, define the *forward* (left) and *backward* (right) messages

$$\boxed{\begin{aligned} \overrightarrow{m}_i(a_i) &\propto \int \mathrm{d}a_{i-1}\, W(a_i \mid a_{i-1})\, \psi_{i-1}(a_{i-1})\, \overrightarrow{m}_{i-1}(a_{i-1}), && \overrightarrow{m}_1 := \mu, \qquad (7.2)\\ \overleftarrow{m}_i(a_i) &\propto \int \mathrm{d}a_{i+1}\, W(a_{i+1} \mid a_i)\, \psi_{i+1}(a_{i+1})\, \overleftarrow{m}_{i+1}(a_{i+1}), && \overleftarrow{m}_n :\equiv 1. \qquad (7.3) \end{aligned}}$$

The base cases matter. $\overrightarrow{m}_1 = \mu$ is the only place the initial law enters; without it the prior on the first site would be silently dropped. $\overleftarrow{m}_n \equiv 1$ says there is no evidence to the right of the last site — an improper flat function, which is fine because it is always multiplied by something integrable.

Proposition 7.1 (Exactness)**.** *With* (7.2)–(7.3)*,*

$$P(a_i \mid x, t) \;\propto\; \overrightarrow{m}_i(a_i)\, \psi_i(a_i)\, \overleftarrow{m}_i(a_i), \tag{7.4}$$

and the pairwise marginal on the edge $(i-1, i)$ *is*

$$P(a_{i-1}, a_i \mid x, t) \;\propto\; \overrightarrow{m}_{i-1}(a_{i-1})\, \psi_{i-1}(a_{i-1})\, W(a_i \mid a_{i-1})\, \psi_i(a_i)\, \overleftarrow{m}_i(a_i). \tag{7.5}$$

Proof. By induction. Unrolling (7.2),

$$\overrightarrow{m}_i(a_i) \propto \int \mathrm{d}a_{1:i-1}\, \mu(a_1) \prod_{j=2}^{i} W(a_j \mid a_{j-1}) \prod_{j=1}^{i-1} \psi_j(a_j),$$

which is exactly the joint density of everything at or left of site i, with the left variables integrated out and ψ_i *not* yet included. Symmetrically $\overleftarrow{m}_i(a_i)$ is the joint over everything strictly right of i, integrated out, given a_i. By Proposition 6.2 these two are conditionally independent given a_i, so their product times the local evidence $\psi_i(a_i)$ is proportional to the marginal. The pairwise statement is the same argument with the separator taken to be the edge rather than a node. □

What goes wrong without this

Note where ψ_i appears. In (7.2) the local evidence at site $i-1$ is absorbed *before* the transition, so $\overrightarrow{m}_i$ carries $\psi_1, \dots, \psi_{i-1}$ and $\overleftarrow{m}_i$ carries $\psi_{i+1}, \dots, \psi_n$. The marginal (7.4) therefore contains each ψ_j exactly once. Absorbing the local evidence *after* the transition instead would put ψ_i into both messages and square it — the belief would be sharper than the truth, and every posterior variance too small. Getting this wrong produces plausible-looking output, which is why the convention is pinned by a test rather than left to a comment.

In the code

`src/bp_grid.py` → `grid_bp` implements (7.2)–(7.4) for one sequence, returning the beliefs and the log-evidence. `src/bp_grid.py` → `grid_bp_batch` does the same for a batch, sharing the site loop so each message update is a single $(K \times K) \cdot (K \times B)$ matrix product. The initialisations are literally `L[0] = exp(log_mu)` and `R[-1] = 1.0`. Pairwise statistics (7.5) are accumulated in `src/em.py` → `e_step`.

7.4 Cost

Each message update integrates one variable against the kernel: on a grid of K points that is a matrix–vector product, $O(K^2)$. There are $2n$ updates, so one sequence costs

$$O(nK^2) \tag{7.6}$$

instead of $O(K^n)$. At $n = 32$, $K = 401$ this is about 5×10^6 operations — microseconds — against the 10^{80} of the naive route.

Intuition

The exponential became linear in the sequence length. Nothing was approximated to achieve it; the saving is entirely from reorganising the order of integration, which the tree structure permits and a general graph does not.

7.5 Reading off the answer

From the belief $b_i(a_i) := P(a_i \mid x, t)$ we get the posterior mean by one more integral,

$$\langle a_i \rangle_{x,t} = \int \mathrm{d}a_i \; a_i \, b_i(a_i), \tag{7.7}$$

and then the score from (4.2). That is the complete pipeline: messages, beliefs, mean, score.

In the code

`src/denoiser.py` → `bp_posterior_mean` is the single entry point used by every experiment — it takes anything exposing `log_transition_matrix`, which covers both the true priors of `src/priors.py` and the estimated kernels of `src/kernels.py`. That shared interface is what makes "exact inference under the true kernel" and "inference under a learned kernel" the same code path with a different argument, so no comparison between them can be confounded by an implementation difference.

7.6 Evidence, and why it is worth carrying

The normalising constant of (7.4) is the *evidence* $P_t(x)$. Messages are renormalised at each step to prevent underflow, and if the discarded constants are accumulated, their product recovers $\log P_t(x)$ exactly.

This is not bookkeeping for its own sake. The evidence is the objective that the estimation of Chapter 10 maximises, and having it in closed form is what makes it possible to check a gradient against a finite difference of the thing being optimised.

In the code

`src/bp_grid.py` → `grid_bp` returns `log_evidence` alongside the beliefs. `src/exact_scores.py` → `gaussian_log_evidence` gives the closed form in the Gaussian case, which is what the general routine is tested against.

7.7 The relationship to forward–backward and to Kalman

Equations (7.2)–(7.3) are the forward–backward algorithm of hidden Markov models, written for a continuous state. If the latent state is discrete the integrals become sums and this is exactly Baum's algorithm; if everything is Gaussian, messages are determined by two numbers each and the recursion becomes the Kalman filter and RTS smoother.

Intuition

These are not three algorithms but one, specialised by what the messages are. Discrete: a vector. Gaussian: a mean and a variance. General continuous: a function, which must be represented somehow — and *that* representation is the only approximation in the whole scheme. Chapter 9 is about it.

7.8 Validation

Two independent checks establish that the implementation computes what this chapter claims.

Against brute force. For a short chain and a coarse grid, the discretised posterior can be enumerated exhaustively — all K^n configurations — and marginalised by summation, using no messages at all. This shares no code with the recursion. Agreement is at machine precision: posterior means to 1.6×10^{-14}, log-evidence to 1.8×10^{-15}, and the pairwise statistic to 3.9×10^{-16}, with the total pairwise mass equal to the edge count exactly.

Against a closed form. On a Gaussian chain the posterior mean is known analytically (4.4), and the recursion reproduces it to 9.2×10^{-15} relative at the working resolution.

In the code

`tests/test_em_bp.py` contains the enumeration check; `tests/test_grid_convergence.py` and `tests/test_score_mean_error_identity.py` contain the Gaussian and identity checks. These tests are the evidence for the numbers above and are the reason those numbers can be quoted at all.

Chapter 8

The Gaussian case and the LMMSE baseline

The Gaussian chain plays two roles: it is the case where everything is computable in closed form, hence the yardstick for validating the general machinery; and it is the model implicitly assumed by the most common approximation, hence the baseline everything is measured against.

8.1 Everything in closed form

Take $\varphi = \mathcal{N}(0, q)$. Then a is jointly Gaussian with covariance $\Sigma_0 = \rho^{|i-j|}$ from (5.4), and by (3.3) so is x, with

$$\Sigma_t \;=\; e^{-2t}\Sigma_0 + \Delta_t I. \tag{8.1}$$

For jointly Gaussian variables the conditional mean is linear, giving (4.4): $\langle a\rangle_{x,t} = e^{-t}\Sigma_0\Sigma_t^{-1}x$ and $S = -\Sigma_t^{-1}x$.

Intuition

The score is a fixed linear map. No inference is required at all — one matrix solve. This is exactly why Gaussian data is a poor probe of how models exploit structure: the answer is linear regression, and every method that can do linear regression is optimal.

8.1.1 The precision matrix is tridiagonal

A useful structural fact. The clean precision $Q_0 = \Sigma_0^{-1}$ of an AR(1) chain is *tridiagonal*: writing the density as $P_0(a) \propto \exp[-\frac{1}{2q}\sum_i (a_i - \rho a_{i-1})^2]$ and expanding, only terms a_i^2 and $a_i a_{i-1}$ appear. So

$$(Q_0)_{ii} = \frac{1+\rho^2}{q} \text{ (interior)}, \qquad (Q_0)_{i,i\pm1} = -\frac{\rho}{q}, \qquad (Q_0)_{ij} = 0 \text{ for } |i-j| > 1.$$

The posterior precision adds the likelihood's contribution, $J = (e^{-2t}/\Delta_t)I + Q_0$, and stays tridiagonal at every t.

Intuition

Tridiagonal precision is the algebraic image of the Markov property: zero in the precision matrix means conditional independence given everything else. The *covariance* Σ_t is dense — every site is correlated with every other — while the precision is banded. Being Markov is a statement about the precision, not the covariance.

8.2 The moment closure

For a non-Gaussian innovation the messages of Chapter 7 are genuinely non-Gaussian functions. A natural approximation keeps only two numbers per message: replace each message by the Gaussian with the same mean and variance. This is cheap, requires no grid, and is what one would write first.

Under the transition $a_i = \rho a_{i-1} + \varepsilon_i$ with ε of mean 0 and variance q, a message with moments (m, v) maps to

$$\text{forward:}\quad (m,v) \mapsto (\rho m,\ \rho^2 v + q); \qquad \text{backward:}\quad (m,v) \mapsto (m/\rho,\ (v+q)/\rho^2), \tag{8.2}$$

the backward map being the forward one inverted, because that recursion integrates over the *output* of the transition rather than its input. It requires $\rho \neq 0$.

Proposition 8.1. *For the chain (5.3) with $\mathbb{E}[a] = 0$ and Gaussian unary likelihoods, the moment-closed recursion returns*

$$\widehat{a}_{\mathrm{LMMSE}} \;=\; e^{-t}\Sigma_0 \left(e^{-2t}\Sigma_0 + \Delta_t I\right)^{-1} x, \tag{8.3}$$

the LMMSE estimator of the covariance-matched Gaussian model, for every *innovation law with those two moments.*

Proof. Both maps in (8.2) depend on φ only through $(0, q)$, and the likelihood absorption is a Gaussian–Gaussian product, which is exact. So the recursion returns the same function of x for every innovation law with those moments — in particular the same as on the covariance-matched *Gaussian* chain. On that chain every message is genuinely Gaussian, so the projection is the identity, the recursion is exact, and it returns the true posterior mean. For jointly Gaussian zero-mean (a, x) the posterior mean is linear and coincides with the LMMSE estimator of Proposition 2.2. □

What goes wrong without this

The zero-mean hypothesis is load-bearing. The general LMMSE estimator is $\mathbb{E}[a] + \mathrm{Cov}(a,x)\,\mathrm{Cov}(x)^{-1}(x - \mathbb{E}[x])$, and dropping the mean terms is only valid when both means vanish. With a prior mean of 1.3, (8.3) departs from the true LMMSE by 0.65 in maximum absolute value.

Intuition

This is a sharper statement than "the Gaussian approximation is approximate". The closure does not merely lose some information about the innovation — it loses *all* of it beyond the variance. Two chains from the matched family of §5.2.2, one Gaussian and one heavily bimodal, are given identical treatment. The approximation is blind by construction, not by degree.

A consequence worth stating: at the single-Gaussian level, "error from representing messages badly" and "error from assuming the wrong model" are the same number. They are different objects only for richer message families.

In the code

`src/bp_gaussian.py` → `gaussian_chain_bp` implements the closure in information form, carrying precision λ and information h rather than mean and variance. That parametrisation matters: a message conveying no information is $\lambda = 0$ exactly, whereas in moment form it is a variance of $+\infty$.

What goes wrong without this

An earlier version of this project implemented the closure by evaluating each Gaussian message *on the grid*, pushing it through the exact update, and re-fitting moments. At weakly informative t the outgoing message is nearly flat, and moment-matching a flat function *truncated* to $[-A, A]$ returns a variance of $(2A)^2/12$ rather than ∞ — an invented precision of order $3/A^2$. The error fed back through the recursion, driving messages towards the boundary. Measured maximum absolute error in the posterior mean at $t = 1.8$: 9.1×10^{-1}, against 4.4×10^{-16} for the information form. The lesson is that the closure must be done analytically; representing an unnormalisable object on a finite grid is not a neutral implementation detail.

8.3 Where the approximation costs something

Since the closure is exactly LMMSE, the gap to exact inference is the value of the higher-order structure. Measured on a Laplace chain at $\rho = 0.85$, the relative deviation of the closure's score from the exact one falls monotonically from 0.198 at $t = 0.08$ to 9.9×10^{-5} at $t = 2.4$.

Intuition

The direction may be surprising: the approximation is *worst at low noise*. The reason is that at large t the likelihood is weak and the posterior is dominated by the smoothing effect of the noise, which Gaussianises everything — so a Gaussian approximation of a nearly Gaussian object is nearly exact. At small t the posterior is sharply peaked and its shape is inherited from the prior, whose non-Gaussianity is then fully visible.
Whether that matters for *generation* is a separate question, because errors at small t enter the reverse process where the denoiser is barely acting. That question is what the experiments of Chapter 11 address, and the answer is not the obvious one.

In the code

`experiments/exp_02_laplace_gaussian_message_error.py` → produces the numbers above; `experiments/exp_03_nongaussian_innovation_sweep.py` → extends them across the innovation families of §5.2.2.

Chapter 9

Representing messages: the numerical layer

Chapter 7 gave an exact algorithm on functions. A computer cannot store a function, so an implementation must choose a representation — and that choice is the *only* approximation in the whole scheme. This chapter is about it, because conflating this approximation with the exactness of the previous chapter is the easiest way to overstate what the method does.

9.1 Three statements that must be kept apart

1. **Sum-product on the posterior chain is exact**, as a theorem about tree-structured factorisations (Proposition 7.1). This is a mathematical statement about functional messages and involves no computer.
2. **The discretised recursion is exact for the discretised model.** Restricting messages to a grid defines a different, finite-state model; on that model the recursion is exact up to floating-point arithmetic.
3. **The discretised model approximates the continuous one**, with an error that must be *measured*.

What goes wrong without this

Writing simply "our method is exact" collapses (1) and (3) and claims something false. Throughout this project we write *exact tree inference* for (1) and *exact for the discretised model* for (2), and never abbreviate. The distinction is not pedantry: (1) holds for any innovation law whatsoever, while the error in (3) depends strongly on the smoothness of that law — as §9.3.2 shows.

9.2 The grid

Messages are represented by their values on K equally spaced points on $[-A, A]$, and integrals become composite trapezoidal sums,

$$\int \mathrm{d}a\, f(a) \;\approx\; \sum_{k=1}^{K} w_k f(u_k), \qquad w_k = \Delta u \text{ (interior)}, \quad \tfrac{1}{2}\Delta u \text{ (endpoints)}. \tag{9.1}$$

Each message update (7.2) becomes a matrix–vector product with the matrix $W(u_l \mid u_k)$, which is formed once per model and reused at every site and every sequence.

In the code

`src/bp_grid.py` → `make_grid` builds the points and trapezoidal weights. The transition matrix comes from each prior's `log_transition_matrix(grid)` (`src/priors.py`) or each estimated kernel's (`src/kernels.py`) — the same interface, which is why the true and learned models are interchangeable in every experiment.

9.3 Two error sources

9.3.1 Truncation

Probability mass outside $[-A, A]$ is discarded. The diagnostic is the mass in the boundary cells: if it is not negligible, A is too small and refining K will not help, because the error is in the domain rather than the resolution.

What goes wrong without this

This failure is invisible unless one looks for it. An earlier package in this project ran at $A = 8$ with a boundary mass of 7.4×10^{-7}, above its own tolerance, and the symptom was not an error message but a slow drift in the posterior means at large t. Widening the domain at preserved spacing reduced it to 5.3×10^{-10}.

9.3.2 Quadrature

The trapezoidal rule has error $O(\Delta u^2)$ for integrands with a bounded second derivative. Whether that bound applies depends on the innovation.

- **Gaussian innovation.** The integrand is smooth and convergence is fast.
- **Laplace innovation.** $\varphi(e) \propto e^{-|e|/b}$ has a discontinuous derivative at $e = 0$. The second derivative is not bounded there, the standard estimate does not apply, and convergence is second order at best.

What goes wrong without this

It is tempting — and this project did it in an earlier draft — to describe the discretisation as "spectrally accurate" on the strength of the Gaussian measurements. That generalisation is not available: spectral accuracy requires smoothness the Laplace kernel does not have. The accompanying note now claims second-order behaviour for that family and nothing more.

9.3.3 A resolution condition

At small t the likelihood $\ell_t(x_i \mid a_i)$ is a narrow Gaussian of width $\sqrt{\Delta_t}/e^{-t}$. If that width is comparable to the grid spacing, the integrand is under-resolved and the quadrature is meaningless however fine the rest looks. The working condition used here is $\sqrt{2t} \gtrsim 3\Delta u$.

Intuition

Everything gets harder as $t \to 0$, from two directions at once: the likelihood narrows, and the score (4.2) carries a $1/\Delta_t$ that diverges. This is why reverse integration must stop at some $t_{\min} > 0$ and why the small-t end of every table deserves the most suspicion.

9.4 Stability

Messages are products of many small numbers and underflow quickly. Two standard devices: work in the log domain where possible with per-row maximum subtraction, and renormalise each message to unit quadrature mass after every step, accumulating the discarded constants so the evidence is recovered exactly.

In the code

Both appear in `src/bp_grid.py` → `grid_bp`: `log_ell -= log_ell.max(...)` before exponentiating, and an explicit mass check that raises `FloatingPointError` if a message loses all its mass rather than silently returning garbage.

9.5 The stopping-time problem

Because integration stops at $t_{\min} > 0$, what a sampler produces is a draw from $P_{t_{\min}}$, not from P_0. This sounds like a technicality and is not.

Independent Gaussian noise of variance v added to a quantity of variance q multiplies its excess kurtosis by $(q/(q+v))^2$. At $\rho = 0.85$ and $t_{\min} = 0.02$ this predicts a generated excess kurtosis of 1.910 where the clean chain has 3.0.

What goes wrong without this

This project measured exactly 1.91 for the reference sampler and initially read it as a failure of the integrator to converge. It was not: the sampler was reproducing $P_{t_{\min}}$ correctly and being compared against the wrong target.

The obvious repair — apply a final denoising step and report $\langle a \rangle_{x,t}$ instead of x — *inverts* the bias rather than removing it. The posterior mean of a heavy-tailed prior is a shrinkage operator: it pulls small values towards zero harder than large ones, so the law of the posterior mean is *more* peaked than P_0. Measured overshoot: 3.73, 4.34, 4.74 at $t_{\min} = 0.02, 0.05, 0.10$ against a true 3.0 — and identical at $K = 401$ and $K = 801$, which rules out grid resolution as the cause.

The correct treatment is to compare like with like: draw from P_0, noise it forward to the same $t_{\min}$, and compare. Both sides are then samples of the same law, no estimator sits between the sampler and the metric, and the target moments follow in closed form.

In the code

`experiments/exp_16_sampling_validation.py` → `reference_at` builds the forward-noised reference; `target_innovation_moments` gives the closed-form target. The docstrings there record both failed designs, since the reasoning is easier to follow than to rediscover.

9.6 Running on a GPU

The recursion is a sequence of dense matrix products, which suits a GPU. Rather than write a second implementation, `src/backend.py` parameterises the existing one by its array module, so CPU and GPU execute the same code.

Intuition

Two implementations of an exact algorithm is a bad trade. They agree on the easy cases and drift where nobody is looking, and the exactness claims of §7.8 would then hold for one device and be assumed for the other.

In the code

`tests/test_backend_parity.py` gates every GPU number: device agreement, the closed-form Gaussian check re-run on device, float64 preservation, and invariance to batch width. Measured on an NVIDIA H200: posterior means agree to 1.8×10^{-16}–8.0×10^{-16}, variances to 9.3×10^{-15}, and the GPU reproduces the closed-form Gaussian score to 4.2×10^{-16}. `hpc/bocconi_gpu.sbatch` → runs that suite before any sweep and exits if it fails.

What goes wrong without this

The availability probe originally tested that a GPU array could be *allocated*. Allocation succeeds without cuBLAS loaded, so on a misconfigured node the check passed and every matrix product then failed inside the recursion. It now probes an actual matrix product. A capability check must exercise the operation that will be used, not a cheaper proxy.

Chapter 10

Estimating the transition kernel

Everything so far assumed the chain is known. This chapter removes that assumption: we observe only noised sequences and must estimate W from them, never seeing a clean sample.

10.1 The objective

Write θ for the kernel parameters. The observed-data log-likelihood is

$$\mathcal{L}(\theta) \;=\; \sum_{\mu=1}^{M} \log P_{t_\mu,\theta}(x^\mu), \qquad P_{t,\theta}(x) = \int \mathrm{d}a\; P_0^\theta(a)\, P(x \mid a, t). \tag{10.1}$$

The integral is the same marginalisation Chapter 7 made tractable — it is the evidence, and BP already returns it.

Intuition

This is a latent-variable problem in the textbook sense: the clean sequence a is the latent variable, the noised x is observed, and the parameters live in the prior. What is unusual is that the latent structure is a tree, so the quantities EM needs are available *exactly* rather than by approximation — which is not the typical situation.

10.2 Expectation maximisation

EM alternates two steps. Given a current $\theta^{(r)}$, form

$$\mathcal{Q}(\theta \mid \theta^{(r)}) = \sum_{\mu} \mathbb{E}_{\theta^{(r)}}\!\left[\log P_\theta(a, x^\mu) \,\middle|\, x^\mu\right], \tag{10.2}$$

then set $\theta^{(r+1)} = \arg\max_\theta \mathcal{Q}$.

Proposition 10.1 (Monotone ascent)**.** $\mathcal{L}(\theta^{(r+1)}) \geq \mathcal{L}(\theta^{(r)})$.

Proof. For any θ, write $\log P_\theta(x) = \mathcal{Q}(\theta \mid \theta^{(r)}) - H(\theta \mid \theta^{(r)})$ with $H(\theta \mid \theta^{(r)}) = \mathbb{E}_{\theta^{(r)}}[\log P_\theta(a \mid x) \mid x]$. By Gibbs' inequality $H(\theta \mid \theta^{(r)}) \leq H(\theta^{(r)} \mid \theta^{(r)})$ for every θ, since the difference is a Kullback–Leibler divergence. Choosing $\theta^{(r+1)}$ to increase $\mathcal{Q}$ therefore increases $\mathcal{L}$ by at least as much. □

Intuition

Monotonicity is the reason EM is trustworthy without a step size. It is also the sharpest available test of an implementation: an error anywhere — in the recursion, the accumulation, or the maximisation — generically breaks it. In every run reported in this project the violation is exactly zero.

10.3 The expectation step is exact

In a general latent-variable model (10.2) is intractable and one resorts to a variational or sampled approximation, losing monotonicity with it. Here the posterior is a tree, so Proposition 7.1 gives the required expectations exactly.

The complete-data log-likelihood is a sum over edges, so the *θ-dependent part* of $\mathcal{Q}$ — it also contains a site-one term and a likelihood term, both θ-free and therefore additive constants — depends on the data only through

$$\boxed{\Xi_{kl} \;=\; \sum_{\mu=1}^{M}\sum_{i=2}^{n} P(a_{i-1}=u_k,\; a_i=u_l \,|\, x^{\mu})\,,} \tag{10.3}$$

the expected transition mass between grid points, accumulated from the pairwise marginals (7.5).

Intuition

Ξ is the continuum analogue of the expected transition counts of Baum–Welch. In the discrete case it literally counts how often the chain was inferred to move from one state to another; here it is a mass on pairs of grid points.

What goes wrong without this

Sufficiency holds only under four conditions: a *homogeneous* kernel shared across sites and sequences, a fixed grid, an initial law excluded from θ, and θ-free likelihood factors. The last is what licenses observations at several noise levels to *add* into one Ξ. Relax any of them and (10.3) is no longer sufficient.

Note also what sufficiency does and does not buy. The maximisation step never revisits the data, so it costs $O(PK^2)$ independently of M. But *forming* Ξ costs $O(MnK^2)$ per iteration and $O(K^2)$ storage. "The whole E-step is one matrix" is a statement about sufficiency, not about inference being cheap.

In the code

`src/em.py` → `e_step` accumulates (10.3); `src/em.py` → `ExpectedStatistics` holds it; `src/em.py` → `fit_em` is the driver with the monotonicity trace. `src/em.py` → `clean_statistics` is the $t \to 0$ limit, which needs no BP at all — the clean-data case Marc's original suggestion referred to.

10.4 Fisher's identity: a gradient without differentiating the recursion

There is a second route to the same information. Fisher's identity states

$$\boxed{\nabla_\theta \mathcal{L}(\theta) = \sum_\mu \mathbb{E}_\theta[\nabla_\theta \log P_\theta(a, x^\mu) \,|\, x^\mu] = \big\langle \Xi(\theta),\ \nabla_\theta \log W_\theta \big\rangle.} \tag{10.4}$$

Proof. $\nabla \log P(x) = \nabla P(x)/P(x)$ and $\nabla P(x) = \int \mathrm{d}a\, \nabla P(a,x) = \int \mathrm{d}a\, P(a,x) \nabla \log P(a,x)$; dividing by $P(x)$ turns $P(a,x)/P(x)$ into the posterior. The second equality follows because $\log P_\theta(a,x)$ depends on θ only through the edge terms. □

Intuition

This says something practically important. BP is not a computation graph to be back-propagated through — it is the oracle that supplies the conditional expectation. The exact gradient of the *marginal* likelihood comes out of one forward–backward pass. Advice received elsewhere on this project held that EM here would require moving the recursion into an automatic-differentiation framework; that is not so, and the entire implementation is plain array arithmetic.

What goes wrong without this

Fisher's identity is *not* EM. It licenses gradient ascent on $\mathcal{L}$; EM maximises $\mathcal{Q}$ at each step. The two are different algorithms and neither dominates. Measured here: on the smooth Gaussian kernel gradient ascent reaches EM's optimum to 2×10^{-9} nats while needing a tuned step size (at too large a step it diverges outright); on the Laplace kernel it attains a *higher* likelihood than EM, because there the exact maximiser of $\mathcal{Q}$ is a lattice artefact of the discretisation.

In the code

`src/em.py` → `q_gradient` implements (10.4); `experiments/exp_08_gradient_vs_exact_mstep.py` → is the comparison. The identity is checked against a central finite difference of the exact evidence in `tests/test_em_bp.py`, agreeing to a relative 1.5×10^{-9}.

10.5 The kernel families

class	free parameters	maximisation step
`GaussianAR1Kernel`	(ρ, q), 2	closed form: belief-weighted Yule–Walker
`LaplaceAR1Kernel`	(ρ, b), 2	closed form, via a weighted median
`MixtureInnovationKernel`	ρ and C triples, $3C$	inner ECM sweeps
`MDNKernel`	network weights	gradient, with backtracking

The mixture kernel is the one used for the headline results,

$$W_\theta(a' \mid a) = \sum_{c=1}^{C} \pi_c \mathcal{N}\big(a' - \rho a;\ \nu_c,\ s_c^2\big)\,, \tag{10.5}$$

because it keeps the linear autoregression while leaving the innovation law essentially free. It has never been shown the density it is asked to recover.

Counting the parameters. At $C = 4$: ρ, four weights, four locations, four scales — thirteen numbers, subject to $\sum_c \pi_c = 1$, hence **twelve free**.

What goes wrong without this

Two subtleties in that count. First, the zero-mean condition $\sum_c \pi_c \nu_c = 0$ is *not* enforced: imposing it by reprojection is not a maximisation step and broke monotone ascent in testing, so it is monitored as a diagnostic instead (it lands at the 10^{-3} level). A reader who assumed the model class of §5.2.2 literally would compute eleven. Second, and consequently, Proposition 8.1 applies to the *true* kernel but not automatically to the estimated one, whose innovation is not exactly centred.
Third: the mixture M-step is not the exact maximiser of $\mathcal{Q}$. The complete-data problem carries a second latent variable — the mixture label — so the step is itself iterative, run as four inner conditional-maximisation sweeps. That makes the algorithm *generalised* EM: monotone ascent is preserved, exact maximisation is not.

In the code

`src/kernels.py` → `MixtureInnovationKernel` — see its `m_step`, whose `n_inner=4` is the ECM sweep count. `theta` exposes the flat parameter vector, which is what the twelve-versus-thirteen count is read from.

10.6 Identifiability

Two claims must be separated, and confusing them overstates what is proved.

Distribution level. Gaussian convolution at finite t is injective. In characteristic-function terms $\varphi_{t,\theta}(\xi) = \varphi_\theta(e^{-t}\xi)\, e^{-\Delta_t \|\xi\|^2/2}$; the Gaussian factor has no zeros so it divides out, and $e^{-t} > 0$ makes $\xi \mapsto e^{-t}\xi$ a bijection. So P_t determines P_0.

Parameter level. That P_0 is determined does *not* by itself determine W. That additionally requires: the initial law fixed or separately identifiable; the transition family identifiable as a parametric family; the mixture treated modulo label permutation; the mixture parameters in the interior — all $\pi_c > 0$ with the (ν_c, s_c) distinct, which an over-specified C need not respect; and μ of full support, since W is pinned down only there. We assume these rather than prove them.

Intuition

Injective is not the same as stable. This is Gaussian deconvolution, whose inverse is unbounded, so identifiability survives at any finite t while the *information* degrades rapidly — and unevenly. Measured per sequence, the Fisher information for the innovation scale falls by a factor 142 between $t = 0.05$ and $t = 1.6$, while that for the correlation falls by 26. The higher-order structure is roughly five times more fragile under the channel than the second-order structure, which is exactly the structure this project cares about.

In the code

`experiments/exp_06_em_parameter_recovery.py` → measures the information decay (`price_of_noising.csv`) and the recovery of an unknown innovation law from noisy data alone.

Chapter 11

Experiments: status and what is settled

This chapter is the current state of the empirical work. It is deliberately explicit about what is finished, what is running, and what has not been started, because several results here are recent and one of them changed the framing of the accompanying note.

11.1 Settled

question	where	finding
Is the recursion right?	`exp_01`, tests	Agrees with brute-force enu 10^{-14} and with the closed-fo to 9.2×10^{-15}.
What does the Gaussian closure cost?	`exp_02`, `exp_03`	Relative score deviation 0.198 to 9.9×10^{-5} at $t = 2.4$; large
Can an unknown innovation law be recovered from noised data?	`exp_06`	Yes. A mixture kernel that Laplace density recovers its exactly and its excess kurtosi C, reaching 2.71 against a tru
Gradient ascent or EM?	`exp_08`	Neither dominates; see §10.
Denoising accuracy against networks	`exp_07`, `replicates`	Ratio 9–14 across seven dou six seeds.
Does a locality-respecting architecture close the gap?	`exp_12`	Most of it. Margin falls fro between 58% and 73% of the work's deficit was its architec
How deep must a network be to run BP on a chain?	`exp_17`	A chain-local network imple message steps to within 1–6 formation horizon, then plate bottoms at depth 6 and rises the representational constrain

11.2 The result that changed the framing

`exp_16` compares the same estimators *through reverse diffusion* rather than pointwise. The validation passes first: run with the true kernel, the sampler reproduces the closed-form target for $P_{t_{\min}}$ to within 1.4% across budgets.

At $M = 2048$ on a Laplace chain, the pointwise and distributional rankings *disagree*. The estimator is an order of magnitude better at denoising and three times better on the covariance profile, while the convolutional network reproduces the innovation law more faithfully by both kurtosis and divergence. The fully connected network shows it most sharply: over the same range its denoising error improves twelvefold while its generated kurtosis degrades from 0.850 to 0.206.

That is the observation. It raises two questions — whether it is an artefact of this particular setting, and what causes it — and §11.3 answers both. The reader who wants the conclusion rather than the sequence can skip there directly; what follows is worth reading only for how the two candidate explanations were separated.

11.3 Resolved since: the dissociation is a capacity effect

Four sweeps have since returned, and together they settle what the dissociation is.

It is not an artefact of the setting. Across the four non-Gaussian families of §5.2.2 at matched covariance, the convolutional network is closer to the target innovation kurtosis than the estimator at $C = 4$ in every case. It persists at $n \in \{32, 64, 128\}$. And it is *identical to three decimals* under grid refinement $K \in \{401, 801, 1601\}$ — the strongest of the three controls, since it removes quadrature as an explanation outright.

It is a limit of the model class, not of the data. Two explanations were available and they differ in consequence. Both the marginal likelihood and the squared-error loss are bulk-dominated, so the tail — little mass, high leverage on shape — is weakly constrained by either; that is a capacity limit and is fixable. Separately, the channel destroys innovation-shape information 142-fold against 26-fold for the correlation (§10), so the data might simply not carry it; that is not fixable.

Varying C separates them:

C	exact	EM–BP	local CNN	global MLP
2	1.921	−0.034	1.273	0.170
4	1.921	0.812	1.273	0.170
8	1.921	**1.319**	1.273	0.170
12	1.921	**1.363**	1.273	0.170
16	1.921	**1.487**	1.273	0.170

(Generated innovation excess kurtosis, $M = 2048$, Laplace chain, median over three seeds, target 1.910. The $C = 4$ entry reads 0.812 here against 0.897 in the four-replicate run quoted earlier; both are the same configuration and the gap is about two standard errors of replicate noise.) The estimator climbs monotonically, is still climbing at $C = 16$, and overtakes the network between $C = 4$ and $C = 8$. So the deficit is **capacity**. The network columns are constant down the table because they do not depend on C — an internal consistency check rather than a finding.

Intuition

The reading that survives is methodological rather than about which method wins. An estimator can be an order of magnitude better at the task it was fitted for and still generate worse samples, because the objective it optimises and the statistic the generated distribution exposes weight the distribution differently. A structured method therefore has to be judged on what it generates, which is exactly the objection that prompted this experiment.

11.4 Still in progress

[PENDING] Nothing above relies on this.

- **Does the richer kernel keep its pointwise advantage?** The component sweep varied C for generation only. If pointwise error degrades as C grows, the two axes trade against each other and the reading changes; if it does not, the estimator dominates on both. Running.

11.5 Known deficiencies in the current measurements

Recorded rather than deferred, because each affects how a number should be read.

- **Error bars: corrected.** An earlier version of this document stated that seeds varied only the initialisation. That was wrong, and checking rather than assuming is what caught it: the replicate index is mixed into every seed that draws training data, so each replicate uses a fresh training set as well as a fresh initialisation. The held-out test set and its exact reference are deliberately common to all replicates. The intervals therefore cover both sampling and optimisation variance, which is the protocol one wants.
- **One hyperparameter was selected on the evaluation set.** The convolutional network's receptive field in `exp_12` was chosen by an oracle over the same held-out set the number is reported on. The bias favours the *baseline*, so the quoted margin is conservative, but the protocol is improper and the run should be repeated with a validation split.
- **Four quantities are asserted only in tests.** The importance-sampling validation, the cross-family closure check, the boundary-mass diagnostic and the baseline-competence figure are checked by the test suite but never written to an output file, so they cannot be independently recomputed from committed data. They are marked as such wherever they appear.
- **The generative comparison covers one family.** Four more are running.

11.6 Not started

- A hybrid estimator combining a learned chain kernel with a learned low-rank correction, which is the natural next step for relaxing the Markov assumption.
- Distillation of the fitted inference-based denoiser into a network, which is the obvious remedy for its two-to-three-hundred-fold inference cost.
- Anything about non-Markov priors. The misspecification limitation of §5 is stated and not addressed.

Bibliography

Marc Mézard. Generative diffusion: updated notes, 2025. Lecture notes, Cargèse summer school.